

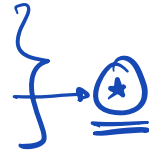
"Hello Everyone!"

Sorting 2

Agenda:

① Insertion Sort

② Quick Sort



(Q) Sort a running integer stream: (inc order)

eg1 $\div$ i/p: 2 10 8 7 9 12 1

sorted: 1 2 7 8 9 10 12

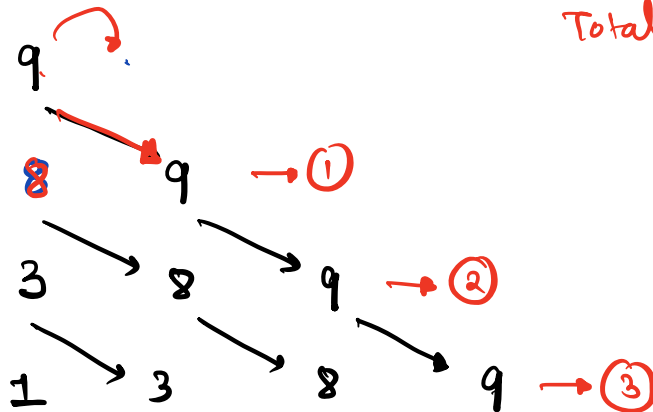


min shifts $\rightarrow$ 0

max shifts $\rightarrow$ no. of elems present.

eg2 $\div$ 9 8 3 1

o/p $\div$



Total shifts:

$$1 + 2 + 3 = \boxed{6}$$

$\Rightarrow$ 4 elem $\rightarrow$ 6 (1+2+3)

n elem $\rightarrow$ 1+2+3+... (n-1)

$$\frac{(n-1)(n-1+1)}{2} \Rightarrow \boxed{\frac{n(n-1)}{2}}$$

$$TC = O(n^2)$$

$$SC = O(1)$$

Pseudocode:

int i
n = 0

for $\forall$ input x

{

for (i = n; i > 0; i--)

{

if (A[i-1] > x)

{

A[i] = A[i-1]

}

else

{

break

}

}

// insert x to its correct position.

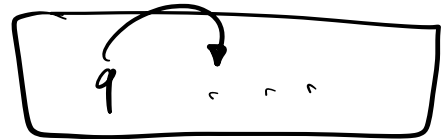
A[i] = x

n++

}

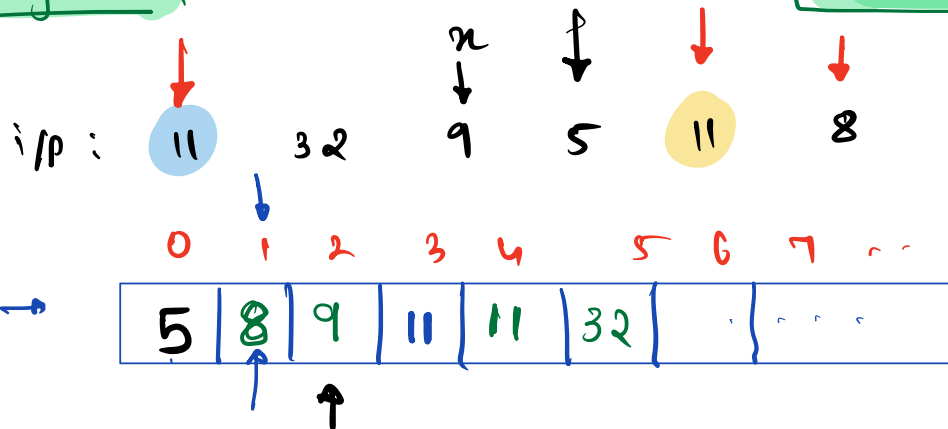
no. of elems already sorted.

elem to be inserted



Dry Run:

*stability.



~~n = 0~~
~~1~~
~~2~~
~~3~~
4

~~i = 0~~

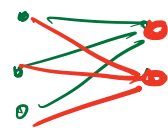
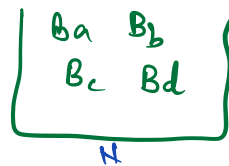
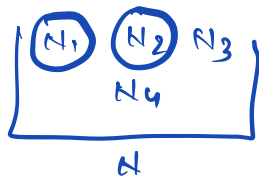
```
int i
n = 0
for (forall input x)
{
    for (i = n; i > 0; i--)
    {
        if (A[i-1] > x)
        {
            A[i] = A[i-1]
        }
        else
        {
            break
        }
    }
    // insert x to the current position
    A[i] = x
    n++
}
```

(8) Given N nuts and N bolts of diff sizes.
 There is a 1:1 mapping b/w nuts and bolts.
Match nuts and bolts efficiently,
 with a constraint that
 comparing a nut with another nut
 or bolt with another bolt
 is NOT allowed.

* Compare nut and a bolt (3 possibilities)

- exact fit
- nut is small
- nut is big

Brute force: Compare all nuts with all bolts
 → $O(n^2)$



* Compare with remaining each time

Logic 2

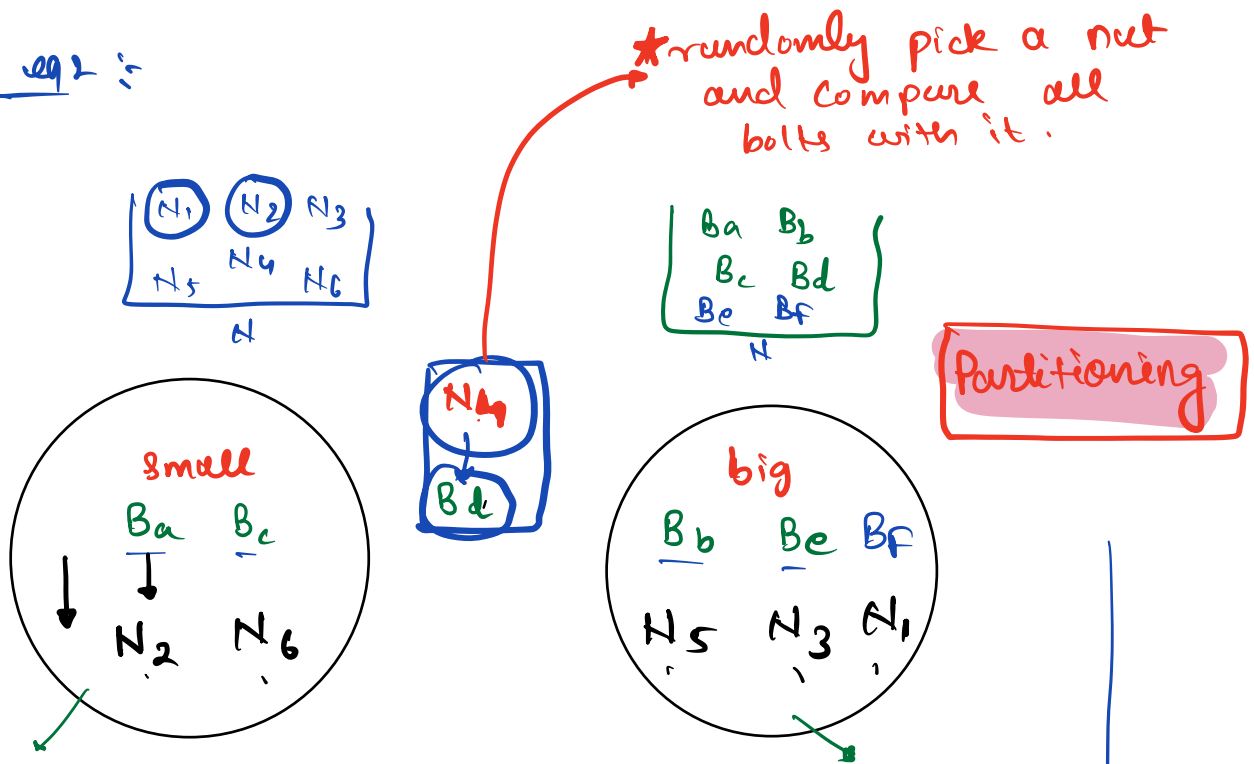
$$\begin{aligned} N_1 &\longrightarrow \underline{n \text{ bolts}} \longrightarrow n \\ N_2 &\longrightarrow \underline{n-1 \text{ bolts}} \longrightarrow (n-1) \\ N_3 &\longrightarrow (n-2) \end{aligned}$$

$$n + (n-1) + (n-2) \dots \dots \dots 1$$

$$TC = O(n^2)$$

↳ Brute Force

eg 2 :-



obs:- Can now compare every remaining nut with Bd (to segregate) bcoz

$N_4 \rightarrow \underline{Bd}$ (fit)

* Refer to below code and problem for implementation.

(Q.2) Given an int array, and x

rearrange the elements

s.t. elements $< x$ are on left side

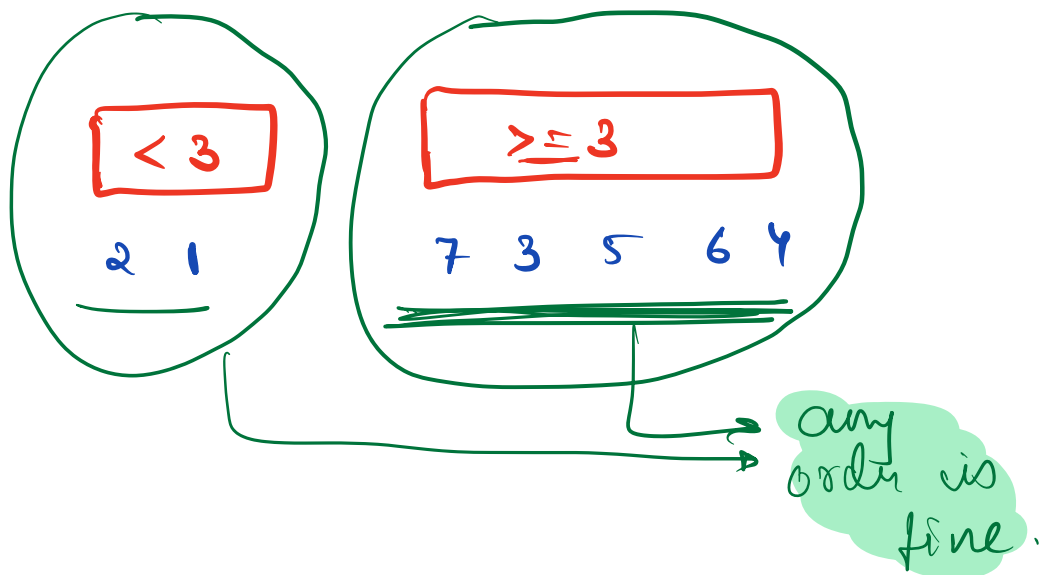
and elements $\geq x$ are on right side.

eg :

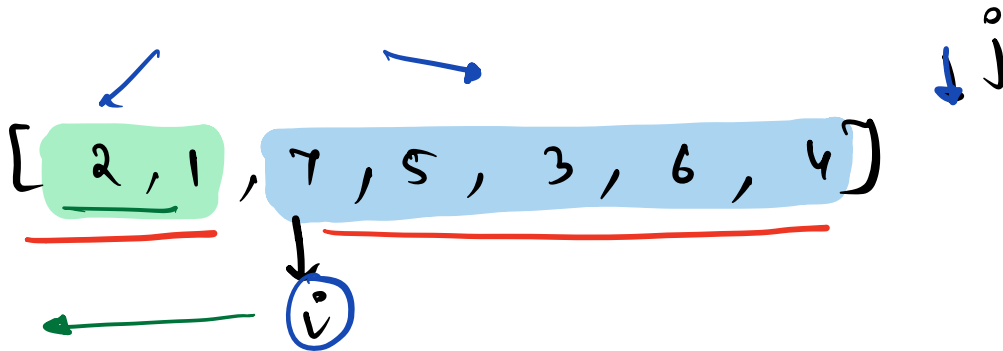
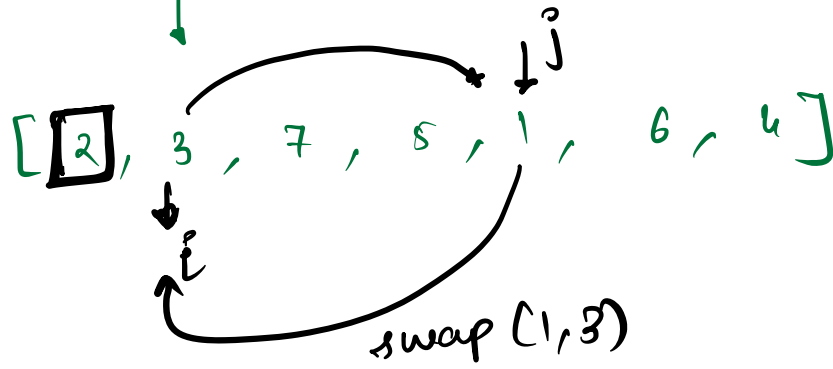
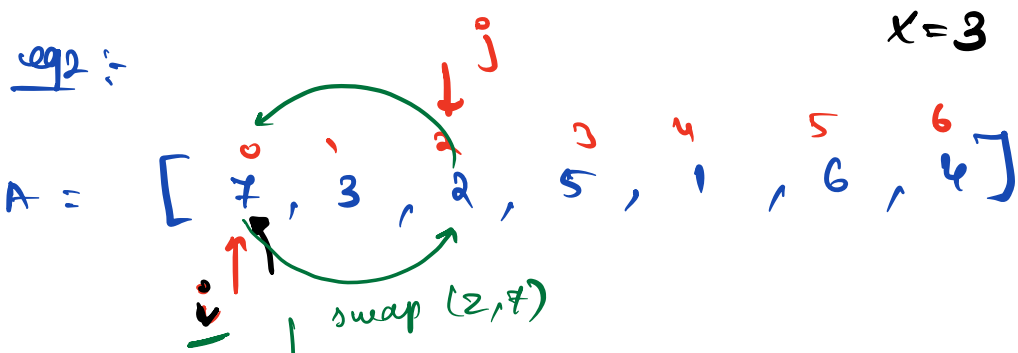
$A = [7, 3, 2, 5, 1, 6, 4]$ ✓

$x = 3$

$SC = O(1)$



① logic $\rightarrow$ Use Insertion Sort $\rightarrow$ $TC: O(n^2)$

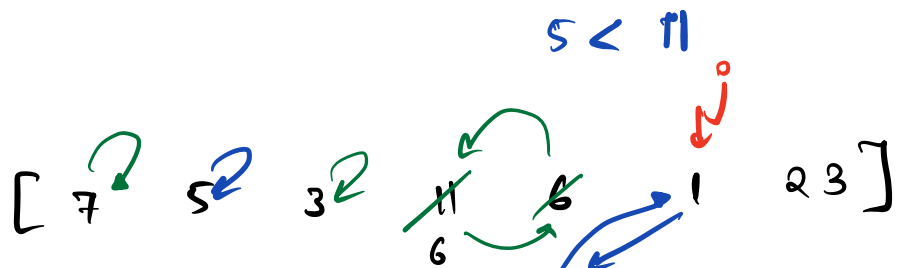


$j \rightarrow$ to iterate over A

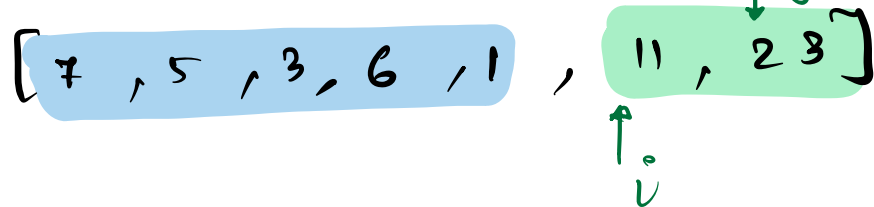
$i \rightarrow$ to track position where next small element will come.

Day Run 2 :

eg :-



$x = 11$



Partitioning Logic

```
i = 0
for (j = 0 ; j < n ; j++)
{
    if (A[j] < x)
    {
        swap(A[i], A[j])
        i++
    }
}
```

$TC = O(n)$
 $SC = O(1)$

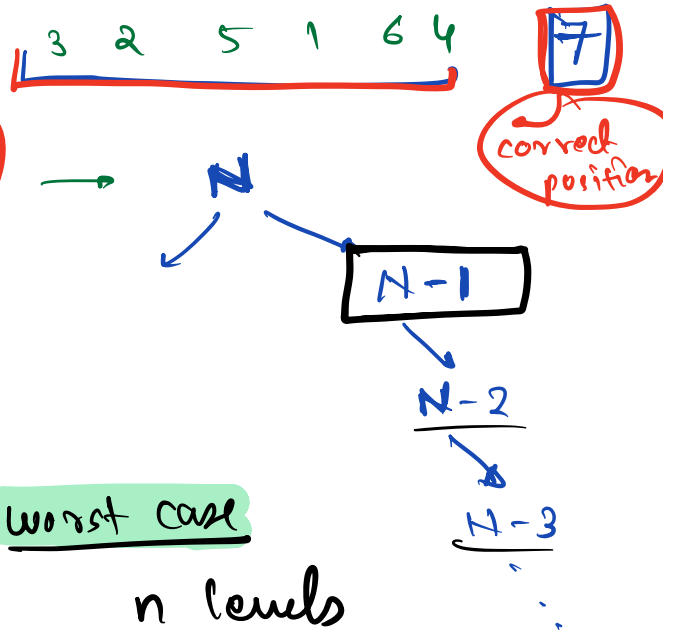
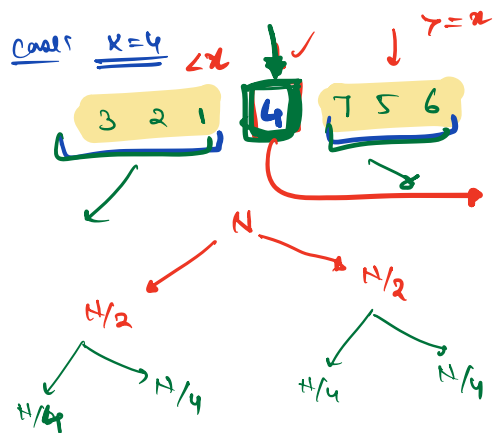
any given int

10:33 → 10:41



eg: $[\overset{\checkmark}{7} \quad 3 \quad \overset{\checkmark}{2} \quad 5 \quad 1 \quad 6 \quad \overset{\checkmark}{\boxed{4}}]$

★ Randomly picking the partition elem x



Best case

$$\text{levels} = \log_2 n$$

$$\text{TC} = O(n * \log_2 n)$$

Worst case

n levels

$$\begin{aligned} \text{TC} &= n * n \\ &= O(n^2) \end{aligned}$$

Best $< \text{TC} < \text{Worst}$



Quick Sort

$$\text{SC} = O(1) + \text{Recursion Stack}$$

↓
partitioning

↓
 $O(\log_2 n)$ ↔ $O(n)$

SC = $O(\log_2 n)$ to $O(n)$

Space Constraint

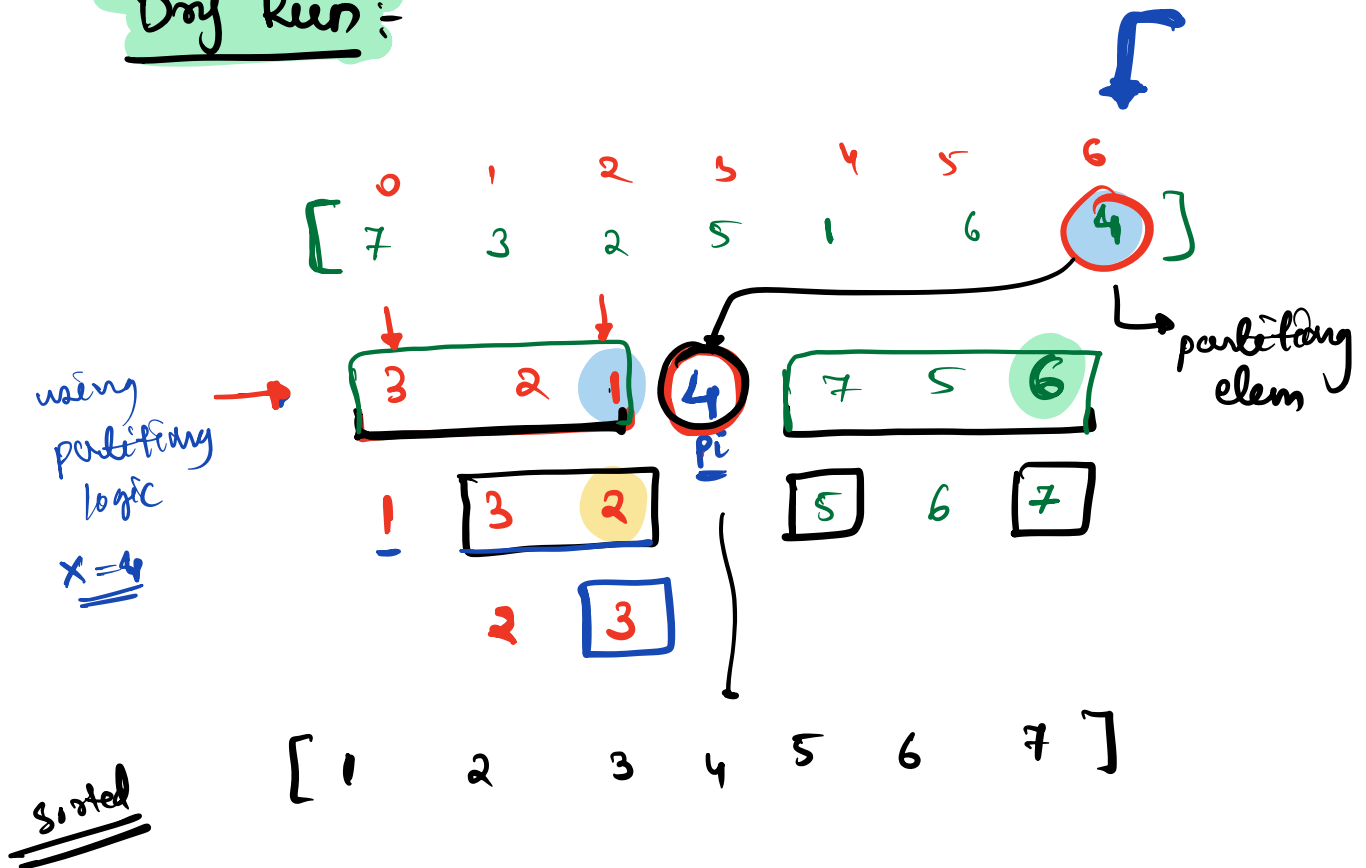
↳ Quick Sort over Merge Sort

Time Constraints

↳ Merge over Quick

↓
* Comparison among Divide and Conquer

Dry Run :-



Dry Run :-

```
void quickSort (A, 0st, n-1end)
{
    // base condition
    if (st >= end)
    {
        return
    }
    correct stores index of the pivot/parting elem
    pi = partition (A, st, end)
```

quickSort (A, st, pi-1)

quickSort (A, pi+1, end)

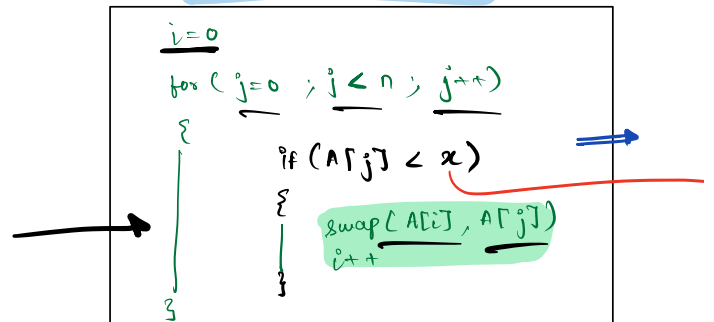
```
int partition (A, st, end)
```

```
{
    // random
    partition - elem = A[end]
    i = st
    for (j = st ; j < end ; j++)
    {
        if (A[j] < partition - elem)
        {
            swap (A[i], A[j])
            i++
        }
    }
    swap (A[i], partition - elem)
    return i
}
```

note

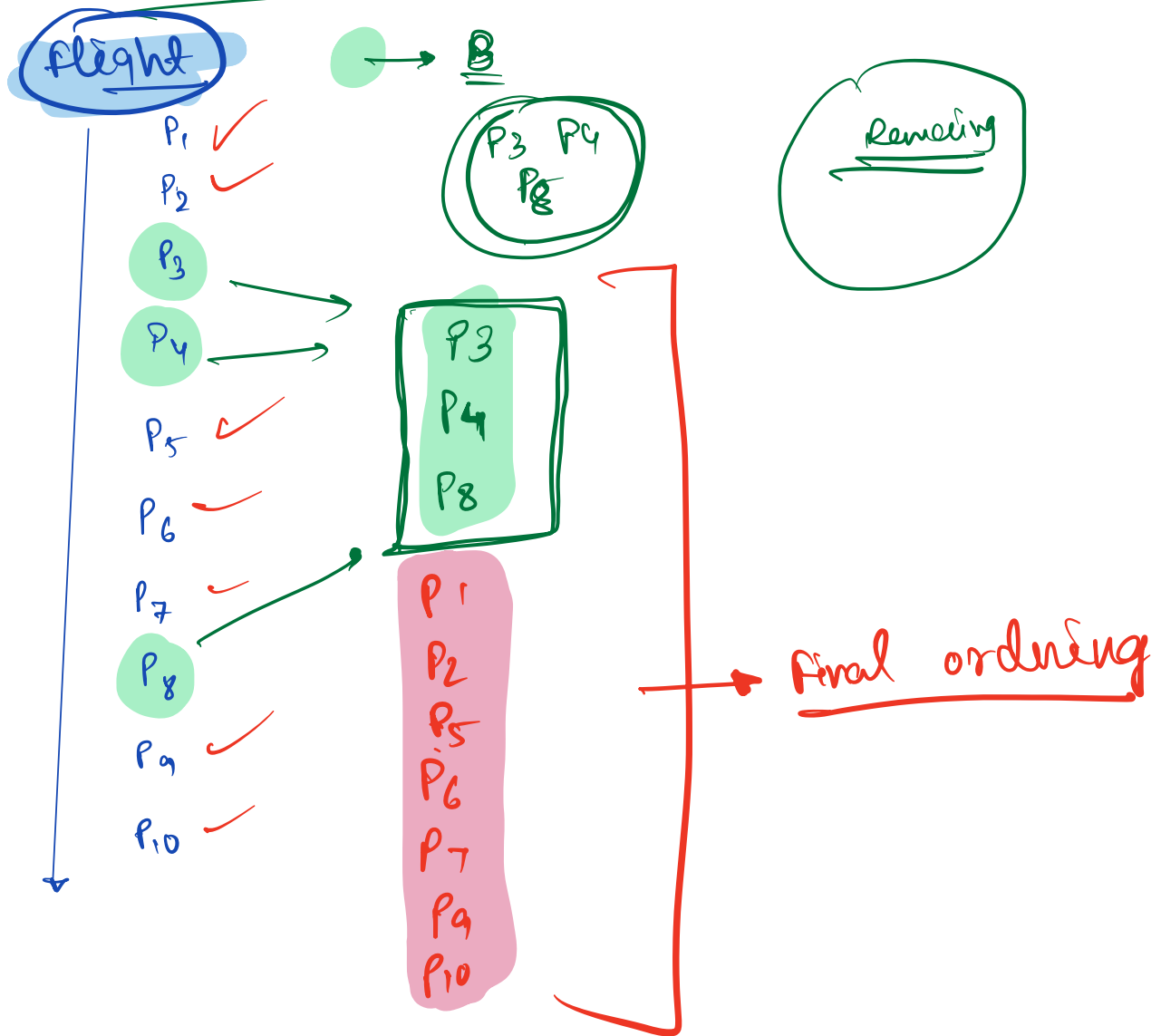
(*) i → correct position in sorted array
end → actual position

Partitioning Logic



* Probabilistic Proof * → next lee

* Stability of a Sorting algorithm



Uber
Pro
example

index		final order
1.	Deepak ✓	③
2.	Nikhil ✓	①
3.	Thakur ✓	④
4.	Madhav ✓	②
5.	Mohsin ✓	⑤

same location

stability in array → if elements are equal

Compare index

